A PROJECT REPORT ON

PocketSmart AI: Your Smart Budget& Recommendation Assistant

Submitted in partial fulfilment of the requirements for the award of the internship

Certificate under

NAAN MUDHALVAN – IBM SKILLSBUILD EMERGING TECHNOLOGIES PROGRAM

Submitted by:

C.ARYA KRISHANA

V.KASTHURI

P.SANDHIYA

M.DHARSHINI

Department of computer science with artificial intelligence

TAMIL NADU SKILL DEVELOPMENT CORPORATION (TNSDC)

IBM INDIA PRIVATE LIMITED

ACADEMIC YEAR 2025 _2026

PocketSmart AI: Your Smart Budget & Recommendation Assistant

Prerequisities

Description

Pre-requisites are the essential tools, software, or knowledge required before starting a project. They ensure the project can be set up, run, and developed smoothly without issues.

Topic Links

Description

1. FastAPI Framework Knowledge: **FastAPI Documentation**

2. **Multi-AI Integration:** API Keys from 10+ providers

3. **Python Async Programming:** Python Asyncio Documentation

4. **Template Engine:** Jinja2 Documentation

5. **HTTP Client Libraries**:HTTPX Documentation

Project Worflow

Description

A **project workflow** is the step-by-step process that guides how tasks move from start to finish.
It ensures work is organized, responsibilities are clear, and goals are achieved efficiently.

Workflow

Description

Epic 1: Gemini AI Setup & Initialization

Story 1: Set up Google Cloud access (or platform) with Gemini 1.5 Flash Pro API access.
Story 2: Configure API keys, usage limits, and test integration for multimodal support (text + image).
Story 3: Fine-tune prompts for budget interpretation and product recommendation generation.

Epic 2: Core Functionality Development

Story 1: Design Flask-based backend structure for routing, session control, and planner modules.
Story 2: Create endpoints for Home Planner, Party Planner, and Jewelry Planner.

Story 3: Integrate Gemini prompts and templates for each planner with support for text + optional image inputs.

Epic 3: Backend Implementation (app.py)

Story 1: Define Flask routes like /generate-home, /generate-party, /generate-jewelry.
Story 2: Organize backend using modular structure:
Story 3: Setup CORS headers for frontend communication and implement basic session management.
Story 4: Include mock API calls or simulated scraping from Amazon, IKEA, Zomato, etc.

Epic 4: UI DevelopmentStory 3:

Story 1: Home Interior Planner (room details, quantities)

Story 2: Party Planner (event type, budget, guests)

Story 3: Jewelry Planner (budget, occasion, outfit image upload)

Story 4: Add dynamic result display and formatting for recommendations using JavaScript.

Epic 5: Testing & Optimization

Story 1: Perform testing with real-world use cases across all three planners.
Story 2: Evaluate Gemini response quality, budget adherence, and platform accuracy.
Story 3: Optimize prompts and add validations to prevent input edge-case failures.
Story 4: Add fallback/default recommendations when AI returns insufficient

Epic 1: Gemini AI Initialization

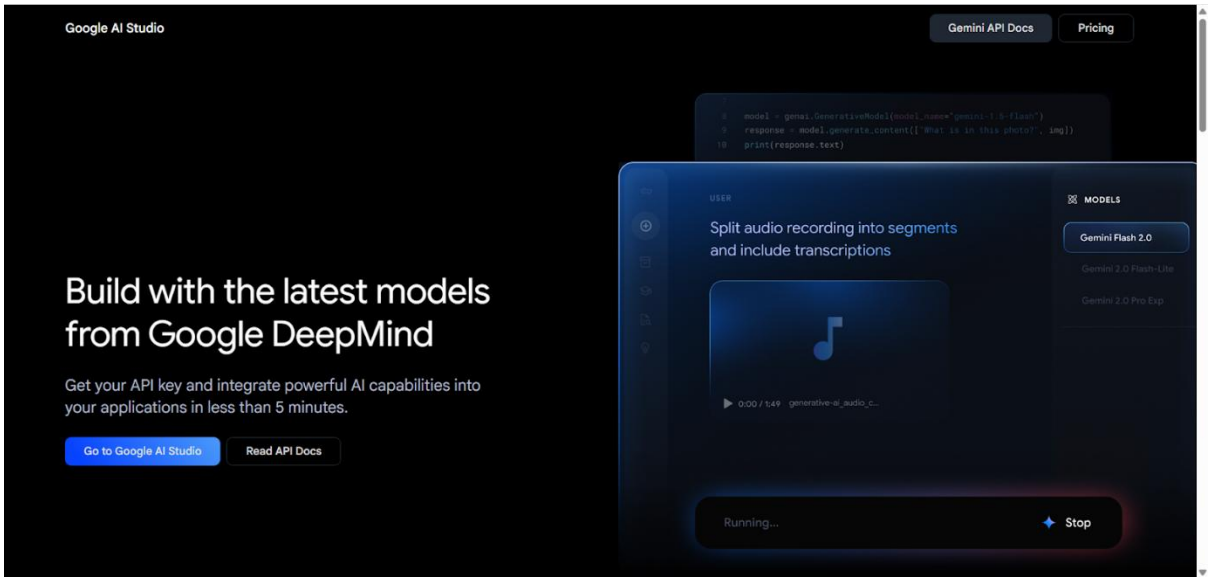
Description

In this foundational stage, the objective is to establish and configure the core AI infrastructure required to power **PocketSmart AI**. The platform relies on **Gemini 1.5 Flash Pro**—Google's multimodal foundation model—to interpret user inputs, process budgets, and generate personalized recommendations. This milestone ensures that the development environment is equipped with authenticated access to Gemini's APIs and is capable of performing both text-based reasoning and image analysis (specifically for the Jewelry Planner).Proper completion of this setup is essential, as it enables all AI-driven functionalities like budget understanding, category-wise recommendations, and multimodal analysis to operate seamlessly.

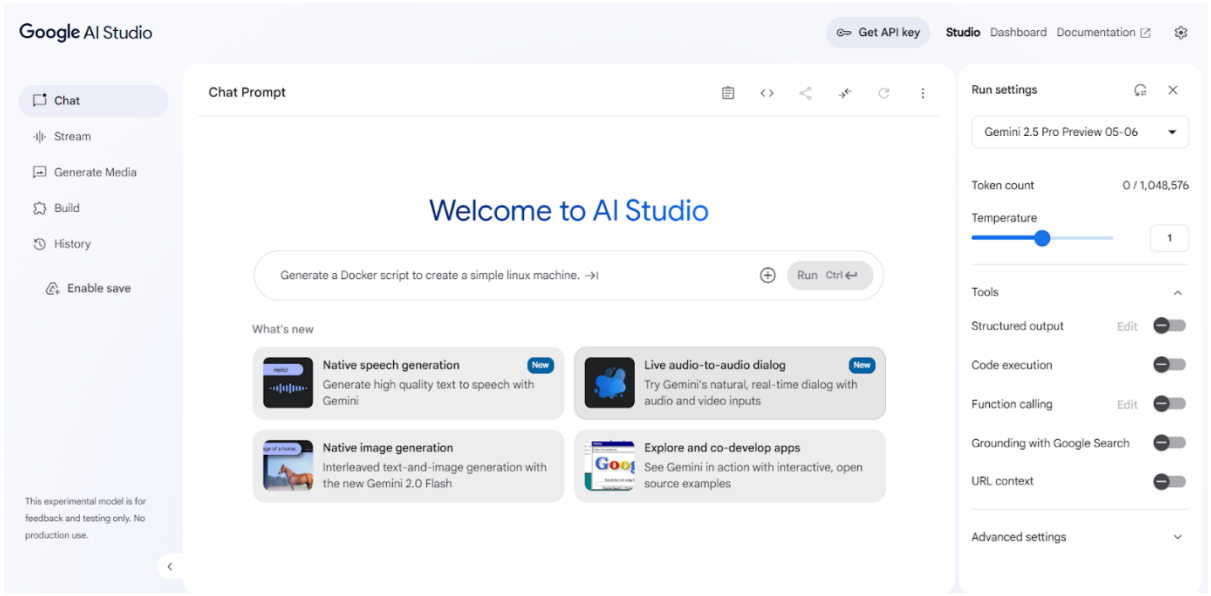
Set up Google Cloud account and enable Gemini API

Description

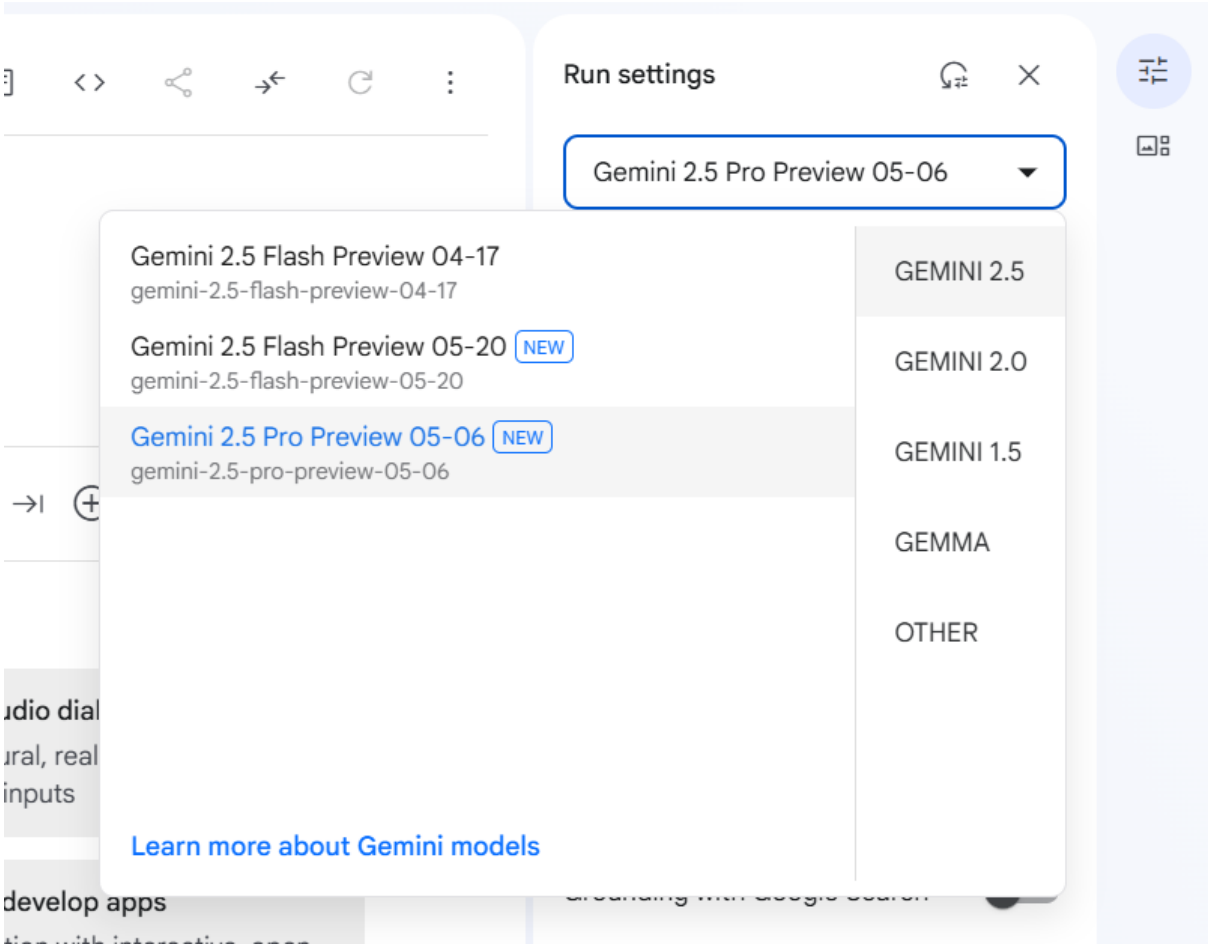
Sign in or create a Google Cloud account via console.cloud.google.com.



Accept all terms and conditions and click on Get API key



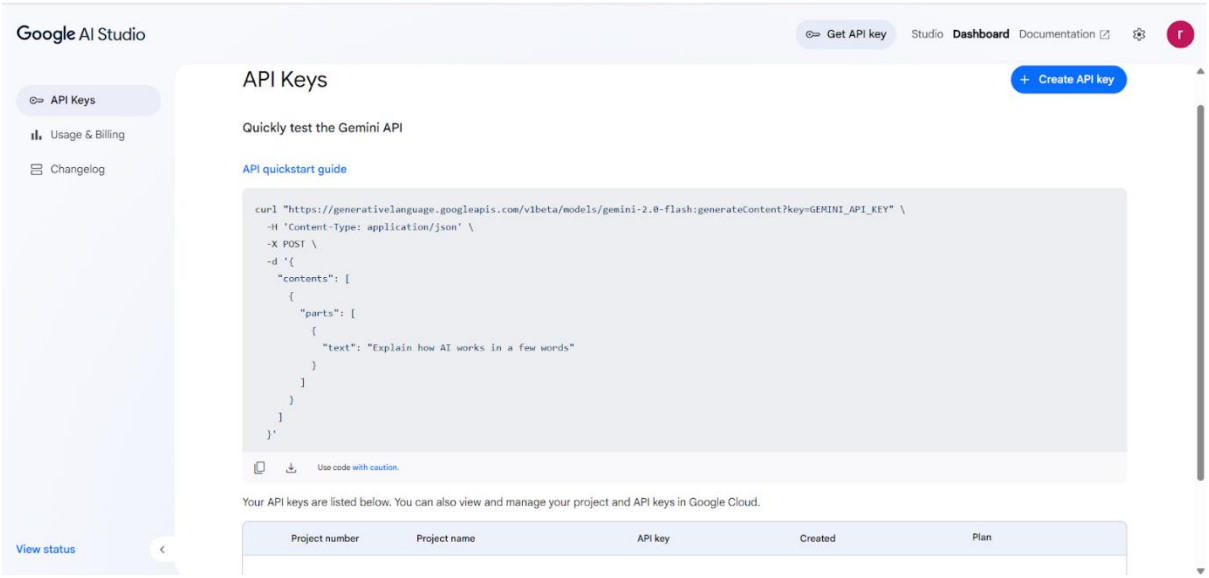
You will get trending models here you can select the model here or you can set the model by go to the API Documentation.



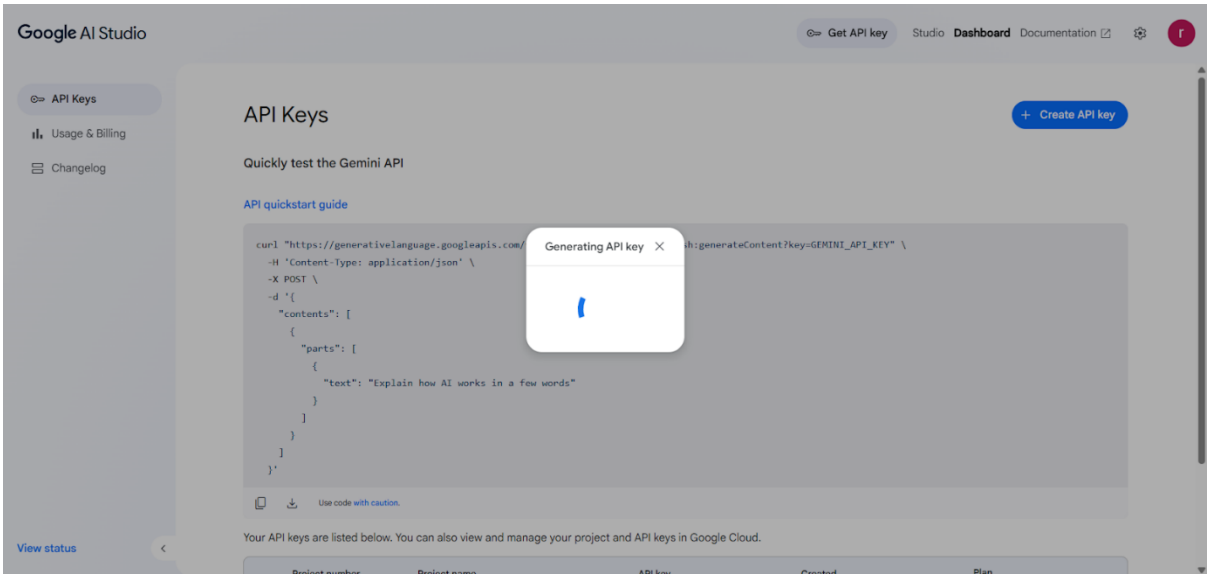
Configure access and permissions

Description

You will get interface like this and now click on Create API key



Once click on API key you will the api key and make sure to copy the API key



Validate Gemini API connectivity

Description

Make a sample prompt call using Python to confirm integration.

```python
from google import genai
from google.genai import types
from PIL import Image
from io import BytesIO
import base64

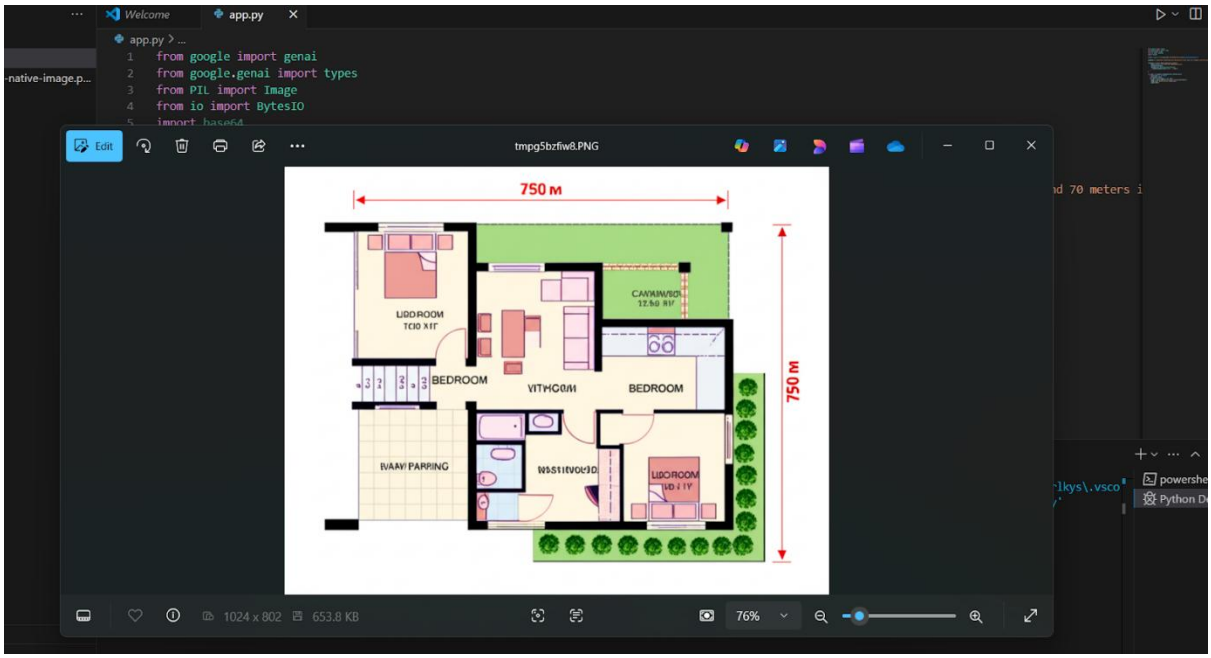
client = genai.Client(api_key="AIzaSyDdzTxAIisLXlQ")

contents = ('Generate a detailed 2D architectural floor plan for a modern two-story house with a total area of 150 meters in length and 70 meters

response = client.models.generate_content(
    model="gemini-2.0-flash-exp-image-generation",
    contents=contents,
    config=types.GenerateContentConfig(
      response_modalities=['Text', 'Image']
    )
)

for part in response.candidates[0].content.parts:
  if part.text is not None:
    print(part.text)
  elif part.inline_data is not None:
    image = Image.open(BytesIO((part.inline_data.data)))
    image.save('gemini-native-image.png')
    image.show()
```

Test both text-only and image + text prompts to ensure multimodal compatibility.



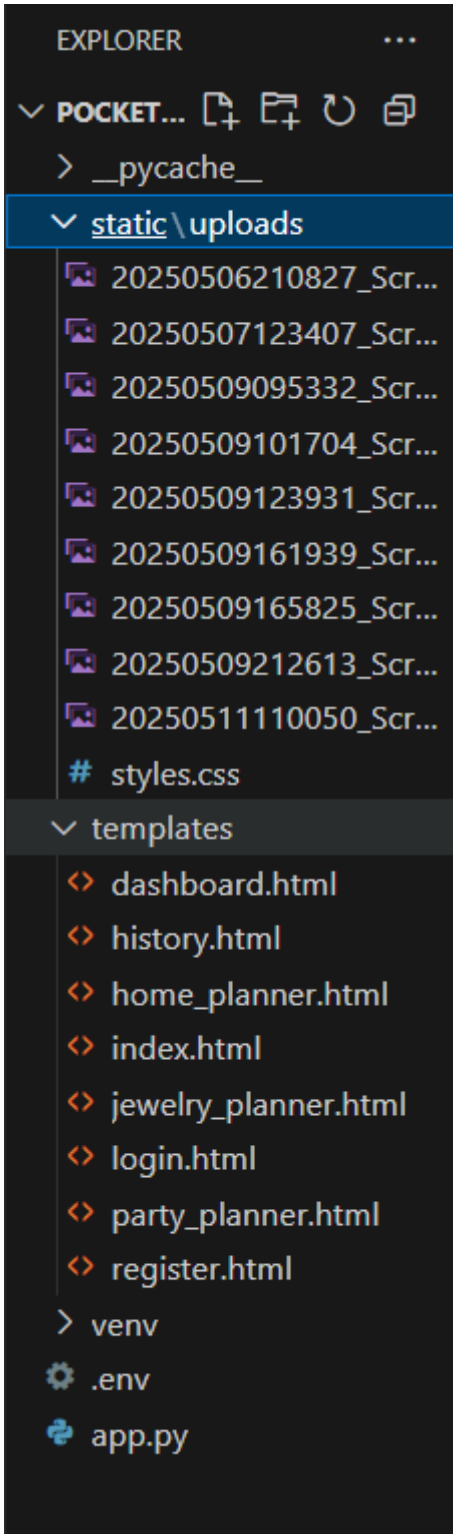
Epic 2: Core Functionalities Development

Description

This focuses on building the **core backend intelligence** of **PocketSmart AI** using the Flask framework and Google's **Gemini 1.5 Flash Pro** foundation model. The primary goal is to develop a robust and modular system capable of accepting user budgets (and optional image inputs), intelligently interpreting the context, and returning domain-

specific, platform-aware recommendations.The backend handles prompt generation, API communication with Gemini, modular planning logic (for Home, Party, and Jewelry), and optional fallback strategies when product matches are limited. All core logic is abstracted into a utility service file (gemini_utils.py) that handles prompt orchestration, budget formatting, domain segmentation, and image analysis for multimodal tasks.

File Explorer Structure



Fast API Instilization and import libraries

Description

Import all required libraries for the project

```
import os
import json
import re
import base64
import shutil
import urllib.parse
from fastapi import FastAPI, HTTPException, Depends, File, UploadFile, Form, Request, status, Cookie
from fastapi.responses import JSONResponse, RedirectResponse
from fastapi.middleware.cors import CORSMiddleware
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
from fastapi.responses import HTMLResponse
from typing import List, Optional, Dict, Any, Union
from pydantic import BaseModel, EmailStr
from passlib.context import CryptContext
from jose import JWTError, jwt
from datetime import datetime, timedelta
from PIL import Image
from io import BytesIO
import google.generativeai as genai
from dotenv import load_dotenv
import asyncio
from typing import Dict, Any, List, Optional
import uuid
```

Fast API Initialization

```
# FastAPI app initialization
app = FastAPI(title="PocketSmart: AI Budget Planner")

SECRET_KEY = os.getenv("SECRET_KEY", "your_secret_key")
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 30
```

Load .env environment

```python
# Load environment variables
load_dotenv()
# Configure Google Gemini AI - Do this once, with consistent environment variable name
API_KEY = os.getenv("GOOGLE_API_KEY")
if not API_KEY:
    # Try alternative environment variable name
    API_KEY = os.getenv("GEMINI_API_KEY")

if not API_KEY:
    raise ValueError("No Google API key found in environment variables. Please set GOOGLE_API_KEY in your .env file.")

genai.configure(api_key=API_KEY)

# Password hashing context
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token", auto_error=False)  # Allow optional token

class RegisterUser(BaseModel):
    username: str
    email: str
    full_name: Optional[str] = None
    password: str

# Configure CORS
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],  # Adjust in production
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Static files and templates
templates = Jinja2Templates(directory="templates")
os.makedirs("static/uploads", exist_ok=True)
app.mount("/static", StaticFiles(directory="static"), name="static")
```

Main Functionality code routings

Description

/generate-home: Generates home interior recommendations (furniture, decor, lighting)

```python
@app.get("/home-planner", response_class=HTMLResponse)
async def home_planner(request: Request, current_user: UserInDB = Depends(get_current_active_user)):
    """Home budget planner page"""
    return templates.TemplateResponse("home_planner.html", {"request": request, "user": current_user})
```

get home_recommendations by using the function

```python
def get_home_recommendations(budget_input: HomeBudgetInput) -> dict:
    """Generate home interior recommendations within budget in INR for Indian market"""
    try:
        # Create prompt for AI model
        prompt = f"""
        I need interior design product recommendations for a home in India with a total budget of ₹{budget_input.total_budget:.2f}.
        Requirements:
        - {budget_input.num_lights} lights/lighting fixtures
        - {budget_input.num_fans} ceiling fans
        - {budget_input.num_furniture} furniture pieces
        - {budget_input.num_dining_tables} dining tables

        Additional rooms to consider:
        {("- Living room" if budget_input.has_living_room else "")}
        {("- Kitchen" if budget_input.has_kitchen else "")}
        {("- Bedroom" if budget_input.has_bedroom else "")}

        Additional requirements: {budget_input.additional_requirements or "None"}

        Please provide a detailed budget breakdown with product recommendations **available in India**.
        Use **Indian brands and pricing**. Include **search terms** suitable for Indian shopping platforms.

        Format your response as JSON with the following structure:
        {{
            "total_budget": {budget_input.total_budget:.2f},
            "budget_breakdown": [
                {{
                    "category": "lighting",
                    "allocation": 0.0,
                    "items": [
                        {{
                            "name": "",
                            "description": "",
                            "estimated_price": 0.0,
                            "quantity": 0,
                            "search_terms": ""
```

linking the websites like amazon,flipkar,ikea ect.... to get the products

```
def get_home_recommendations(budget_input: HomeBudgetInput) -> dict:
                    }}
                ]
            }}
        ],
        "calculation_table": [
            {{
                "category": "",
                "items_count": 0,
                "total_cost": 0.0,
                "percentage_of_budget": 0.0
            }}
        ],
        "remaining_budget": 0.0,
        "additional_suggestions": []
    }}

    Ensure total costs stay within budget. Include search terms for each item to find on shopping websites like Flipkart, Amazon India, IKEA I
    """

    response = model.generate_content(prompt)
    result = extract_json_from_response(response.text)

    # Add shopping links for each item
    for category in result.get("budget_breakdown", []):
        for item in category.get("items", []):
            search_terms = item.get("search_terms", "")
            if search_terms:
                item["shopping_links"] = {
                    "amazon": f"https://www.amazon.in/s?k={urllib.parse.quote_plus(search_terms)}",
                    "flipkart": f"https://www.flipkart.com/search?q={urllib.parse.quote_plus(search_terms)}",
                    "ikea": f"https://www.ikea.com/in/en/search/?q={urllib.parse.quote_plus(search_terms)}",
                    "myntra": f"https://www.myntra.com/search?q={urllib.parse.quote_plus(search_terms)}",
                    "ajio": f"https://www.ajio.com/search/?text={urllib.parse.quote_plus(search_terms)}"
                }

    return result
```

```
    except Exception as e:
        raise HTTPException(500, f"Error generating recommendations: {str(e)}")


def usd_to_inr(amount_usd: float, exchange_rate: float = 83.0) -> float:
    """Convert USD amount to INR using the specified exchange rate"""
    return amount_usd * exchange_rate
```

/generate-party: Suggests catering, venue, and decoration options for events

```
@app.get("/party-planner", response_class=HTMLResponse)
async def party_planner(request: Request, current_user: UserInDB = Depends(get_current_active_user)):
    """Party budget planner page"""
    return templates.TemplateResponse("party_planner.html", {"request": request, "user": current_user})
```

get home_recommendations by using the function

```python
def get_party_recommendations(budget_input: PartyBudgetInput) -> dict:
    """Generate party planning recommendations within budget in INR for Indian market"""
    try:
        # Create prompt for AI model
        prompt = f"""
        I need party planning recommendations for India with a total budget of ₹{budget_input.total_budget:.2f}.

        Party details:
        - Type: {budget_input.party_type}
        - Number of guests: {budget_input.num_guests}
        - Venue type: {budget_input.venue_type or "Not specified"}
        - Catering needed: {"Yes" if budget_input.needs_catering else "No"}
        - Decoration needed: {"Yes" if budget_input.needs_decoration else "No"}
        - Entertainment needed: {"Yes" if budget_input.needs_entertainment else "No"}

        Additional requirements: {budget_input.additional_requirements or "None"}

        Please provide a detailed budget breakdown with specific recommendations available in India using INR prices.
        Use Indian brands, services, and typical cost expectations.

        Format your response as JSON with the following structure:
        {{
            "total_budget": {budget_input.total_budget:.2f},
            "budget_breakdown": [
                {{
                    "category": "venue",
                    "allocation": 0.0,
                    "items": [
                        {{
                            "name": "",
                            "description": "",
                            "estimated_price": 0.0,
                            "quantity": 0,
                            "search_terms": ""
                        }}
                    ]
                }}
```

```
def get_party_recommendations(budget_input: PartyBudgetInput) -> dict:
            ],
            "venue_suggestions": [
                {{
                    "name": "",
                    "type": "",
                    "capacity": 0,
                    "estimated_cost": 0.0,
                    "search_terms": ""
                }}
            ],
            "remaining_budget": 0.0,
            "additional_suggestions": []
        }}

        Ensure all costs are in INR and total does not exceed the given budget.
        Provide search terms suitable for Indian websites such as BookMyShow, Swiggy, Flipkart, etc.
        """

        response = model.generate_content(prompt)
        result = extract_json_from_response(response.text)

        # Create INR calculation table
        result["calculation_table_inr"] = []
        categories = {}

        for category in result.get("budget_breakdown", []):
            cat_name = category.get("category", "Misc")
            cat_allocation = category.get("allocation", 0)

            if cat_name not in categories:
                categories[cat_name] = {
                    "category": cat_name,
                    "items_count": 0,
                    "total_cost": 0,
                    "percentage_of_budget": 0
                }
```

linking the websites like amazon,flipkar,ikea ect.... to get the products

```python
def get_party_recommendations(budget_input: PartyBudgetInput) -> dict:
            for item in category.get("items", []):
                categories[cat_name]["items_count"] += 1
                categories[cat_name]["total_cost"] += item.get("estimated_price", 0)

            if result["total_budget"] > 0:
                categories[cat_name]["percentage_of_budget"] = (categories[cat_name]["total_cost"] / result["total_budget"]) * 100

        for cat_data in categories.values():
            result["calculation_table_inr"].append(cat_data)

        # Define relevant shopping platforms for each category
        category_platforms = {
            "venue": ["google", "booking", "makemytrip", "oyorooms", "nobroker"],
            "catering": ["swiggy", "zomato"],
            "food": ["swiggy", "zomato", "bigbasket", "amazon", "flipkart"],
            "drinks": ["swiggy", "zomato", "bigbasket", "amazon", "flipkart"],
            "decoration": ["amazon", "flipkart", "meesho", "myntra"],
            "entertainment": ["bookmyshow", "amazon", "flipkart"],
            "gifts": ["amazon", "flipkart", "myntra", "meesho"],
            "photography": ["google", "amazon", "flipkart"],
            "music": ["amazon", "flipkart", "bookmyshow"],
            "games": ["amazon", "flipkart"],
            "accessories": ["amazon", "flipkart", "myntra", "meesho"],
            "transportation": ["makemytrip", "google"],
            "return_gifts": ["amazon", "flipkart", "myntra", "meesho"]
        }

        # Default platforms if category is not in our predefined list
        default_platforms = ["amazon", "flipkart", "google"]

        # Add shopping links for each item
        for category in result.get("budget_breakdown", []):
            cat_name = category.get("category", "").lower()
            relevant_platforms = category_platforms.get(cat_name, default_platforms)

            for item in category.get("items", []):
```

using the if condition to categorize the situation based products

```python
def get_party_recommendations(budget_input: PartyBudgetInput) -> dict:
        for category in result.get("budget_breakdown", []):
            cat_name = category.get("category", "").lower()
            relevant_platforms = category_platforms.get(cat_name, default_platforms)

            for item in category.get("items", []):
                search_terms = item.get("search_terms", "")
                if search_terms:
                    item["shopping_links"] = {}

                    # Only add relevant platforms for this category
                    if "amazon" in relevant_platforms:
                        item["shopping_links"]["amazon"] = f"https://www.amazon.in/s?k={urllib.parse.quote_plus(search_terms)}"
                    if "flipkart" in relevant_platforms:
                        item["shopping_links"]["flipkart"] = f"https://www.flipkart.com/search?q={urllib.parse.quote_plus(search_terms)}"
                    if "bigbasket" in relevant_platforms:
                        item["shopping_links"]["bigbasket"] = f"https://www.bigbasket.com/ps/?q={urllib.parse.quote_plus(search_terms)}"
                    if "swiggy" in relevant_platforms:
                        item["shopping_links"]["swiggy"] = f"https://www.swiggy.com/search?query={urllib.parse.quote_plus(search_terms)}"
                    if "zomato" in relevant_platforms:
                        item["shopping_links"]["zomato"] = f"https://www.zomato.com/search?q={urllib.parse.quote_plus(search_terms)}"
                    if "bookmyshow" in relevant_platforms:
                        item["shopping_links"]["bookmyshow"] = f"https://in.bookmyshow.com/search?q={urllib.parse.quote_plus(search_terms)}"
                    if "myntra" in relevant_platforms:
                        item["shopping_links"]["myntra"] = f"https://www.myntra.com/search?q={urllib.parse.quote_plus(search_terms)}"
                    if "meesho" in relevant_platforms:
                        item["shopping_links"]["meesho"] = f"https://www.meesho.com/search?q={urllib.parse.quote_plus(search_terms)}"
                    if "google" in relevant_platforms:
                        item["shopping_links"]["google"] = f"https://www.google.com/search?q={urllib.parse.quote_plus(search_terms)}"
                    if "booking" in relevant_platforms:
                        item["shopping_links"]["booking"] = f"https://www.booking.com/search.html?ss={urllib.parse.quote_plus(search_terms)}"
                    if "makemytrip" in relevant_platforms:
                        item["shopping_links"]["makemytrip"] = f"https://www.makemytrip.com/hotels/hotel-listing/?searchText={urllib.parse.quote
                    if "oyorooms" in relevant_platforms:
                        item["shopping_links"]["oyorooms"] = f"https://www.oyorooms.com/search/?location={urllib.parse.quote_plus(search_terms)}"
                    if "nobroker" in relevant_platforms:
                        item["shopping_links"]["nobroker"] = f"https://www.nobroker.in/property/search?searchTerm={urllib.parse.quote_plus(search
```

```python
    # Add search links for venue suggestions
    venue_platforms = ["google", "booking", "makemytrip", "oyorooms", "nobroker"]
    for venue in result.get("venue_suggestions", []):
        search_terms = venue.get("search_terms", "")
        if search_terms:
            venue["search_links"] = {}
            if "google" in venue_platforms:
                venue["search_links"]["google"] = f"https://www.google.com/search?q={urllib.parse.quote_plus(search_terms)}"
            if "booking" in venue_platforms:
                venue["search_links"]["booking"] = f"https://www.booking.com/search.html?ss={urllib.parse.quote_plus(search_terms)}"
            if "makemytrip" in venue_platforms:
                venue["search_links"]["makemytrip"] = f"https://www.makemytrip.com/hotels/hotel-listing/?searchText={urllib.parse.quote_plus(s
            if "oyorooms" in venue_platforms:
                venue["search_links"]["oyorooms"] = f"https://www.oyorooms.com/search/?location={urllib.parse.quote_plus(search_terms)}"
            if "nobroker" in venue_platforms:
                venue["search_links"]["nobroker"] = f"https://www.nobroker.in/property/search?searchTerm={urllib.parse.quote_plus(search_terms

    return result

except Exception as e:
    raise HTTPException(500, f"Error generating recommendations: {str(e)}")
```

/generate-jewelry: Accepts text + image inputs to return style-matched jewelry options.

async def jewelry_planner(request: Request, current_user: UserInDB = Depends(get_current_active_user)):

"""Jewelry budget planner page"""

return templates.TemplateResponse("jewelry_planner.html", {"request": request, "user": current_user})

get home_recommendations by using the function

```python
def get_jewelry_recommendations(budget_input: JewelryBudgetInput, image_path: Optional[str] = None) -> dict:
    """Generate jewelry recommendations based on uploaded dress and budget in INR (India-specific)"""
    try:
        # Base prompt with INR context
        base_prompt = f"""
        I need jewelry recommendations for India with a total budget of ₹{budget_input.total_budget:.2f}.

        Occasion: {budget_input.occasion}
        Preferences: {budget_input.preferences or "Not specified"}
        Provide only India-relevant styles, availability, and price ranges in INR.
        """

        if image_path:
            # Include image-based outfit analysis
            img = Image.open(image_path)
            prompt = base_prompt + """

            An image of the outfit is uploaded. Suggest jewelry that complements it, considering color, design, and occasion appropriateness.

            Format the output as JSON:
            {
                "outfit_analysis": {
                    "colors": [],
                    "style": "",
                    "formality": ""
                },
                "total_budget": 0.0,
                "jewelry_recommendations": [
                    {
                        "item_type": "",
                        "description": "",
                        "style": "",
                        "estimated_price": 0.0,
                        "search_terms": ""
                    }
                ],
                "remaining_budget": 0.0,
```

```python
def get_jewelry_recommendations(budget_input: JewelryBudgetInput, image_path: Optional[str] = None) -> dict:
                "styling_tips": []
            }

            Make sure prices are in INR and stay within budget.
            Include Indian-friendly search terms for shopping.
            """
            response = model.generate_content([prompt, img])
        else:
            # Only text input
            prompt = base_prompt + """

            Format the output as JSON:
            {
                "total_budget": 0.0,
                "jewelry_recommendations": [
                    {
                        "item_type": "",
                        "description": "",
                        "style": "",
                        "estimated_price": 0.0,
                        "search_terms": ""
                    }
                ],
                "remaining_budget": 0.0,
                "styling_tips": []
            }

            Keep prices in INR and relevant to Indian brands.
            """
            response = model.generate_content(prompt)

        result = extract_json_from_response(response.text)

        # Add shopping links for each item (India-specific)
        for item in result.get("jewelry_recommendations", []):
            search_terms = item.get("search_terms", "")
```

linking the websites like amazon,flipkar,ikea ect.... to get the products

```python
def get_jewelry_recommendations(budget_input: JewelryBudgetInput, image_path: Optional[str] = None) -> dict:
        # Add shopping links for each item (India-specific)
        for item in result.get("jewelry_recommendations", []):
            search_terms = item.get("search_terms", "")
            if search_terms:
                item["shopping_links"] = {
                    "amazon": f"https://www.amazon.in/s?k={urllib.parse.quote_plus(search_terms)}",
                    "flipkart": f"https://www.flipkart.com/search?q={urllib.parse.quote_plus(search_terms)}",
                    "bluestone": f"https://www.bluestone.com/search.html?query={urllib.parse.quote_plus(search_terms)}",
                    "tanishq": f"https://www.tanishq.co.in/search?q={urllib.parse.quote_plus(search_terms)}",
                    "caratlane": f"https://www.caratlane.com/search?q={urllib.parse.quote_plus(search_terms)}",
                    "melorra": f"https://www.melorra.com/search?q={urllib.parse.quote_plus(search_terms)}",
                    "meesho": f"https://www.meesho.com/search?q={urllib.parse.quote_plus(search_terms)}"
                }

        return result

    except Exception as e:
        raise HTTPException(500, f"Error generating recommendations: {str(e)}")
```

Login and Register page routings

Description

Login page

/login: Authenticates user credentials and initiates a session upon successful login.

```python
@app.get("/login", response_class=HTMLResponse)
async def login_page(request: Request):
    """Serve the login page"""
    # Check if user is already logged in
    try:
        token = await get_token(request)
        if token:
            user = await get_current_user(request, token)
            if user:
                return RedirectResponse(url="/dashboard", status_code=status.HTTP_302_FOUND)
    except Exception:
        pass

    return templates.TemplateResponse("login.html", {"request": request})
```

Register Page

/register: Handles new user registration by accepting and securely storing user credentials.

```python
@app.get("/register", response_class=HTMLResponse)
async def register_page(request: Request):
    """Serve the registration page"""
    # Check if user is already logged in
    try:
        token = await get_token(request)
        if token:
            user = await get_current_user(request, token)
            if user:
                return RedirectResponse(url="/dashboard", status_code=status.HTTP_302_FOUND)
    except Exception:
        pass

    return templates.TemplateResponse("register.html", {"request": request})
```

Logout Page

/logout: Terminates the current user session and redirects to the login page.

```python
@app.post("/logout")
async def logout(request: Request):
    """Logout user by blacklisting their token and clearing session"""
    token = await get_token(request)

    if token:
        # Add token to blacklist
        blacklisted_tokens.add(token)

        try:
            # Extract username from token to remove from active sessions
            payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
            username = payload.get("sub")
            if username and username in active_sessions:
                del active_sessions[username]
        except JWTError:
            pass

    # Use RedirectResponse instead of JSONResponse for proper redirection
    response = RedirectResponse(url="/login", status_code=status.HTTP_302_FOUND)
    response.delete_cookie(key="access_token")
```

Ensure modular code structure

Description

/token: Issues a secure JWT token after successful authentication, enabling authorized access to protected API routes.

```python
@app.post("/token", response_model=Token)
async def login_for_access_token(form_data: OAuth2PasswordRequestForm = Depends()):
    """Login endpoint to get access token"""
    user = authenticate_user(users_db, form_data.username, form_data.password)
    if not user:
        raise HTTPException(
            status_code=401,
            detail="Incorrect username or password",
            headers={"WWW-Authenticate": "Bearer"},
        )

    # Create access token
    access_token_expires = timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    access_token = create_access_token(
        data={"sub": user.username}, expires_delta=access_token_expires
    )

    # Create or update session for the user
    existing_user_data = {}
    if user.username in active_sessions:
        existing_user_data = active_sessions[user.username].user_data
        # If there's an old token, blacklist it
        old_token = active_sessions[user.username].token
        blacklisted_tokens.add(old_token)

    active_sessions[user.username] = UserSession(
        username=user.username,
        login_time=datetime.utcnow(),
        last_activity=datetime.utcnow(),
        token=access_token,
        user_data=existing_user_data
    )

    # Return response with cookie
    response = JSONResponse(content={"access_token": access_token, "token_type": "bearer"})

    response.set_cookie(
        key="access_token",
        value=access_token,  # Store token directly without Bearer prefix
        httponly=True,
        max_age=ACCESS_TOKEN_EXPIRE_MINUTES * 60,
        samesite="lax"
    )

    return response
```

session-info: Retrieves metadata about the current user session, such as user ID and login status.

/session-data: Returns detailed session-specific data used for personalization and recommendation tracking

```python
@app.get("/session-info")
async def get_session_info(request: Request, current_user: UserInDB = Depends(get_current_active_user)):
    """Get current user's session information"""
    if current_user.username in active_sessions:
        session = active_sessions[current_user.username]
        return {
            "username": session.username,
            "login_time": session.login_time,
            "last_activity": session.last_activity,
            "session_duration": (datetime.utcnow() - session.login_time).total_seconds() // 60,  # in minutes
            "user_data": session.user_data
        }
    else:
        raise HTTPException(status_code=404, detail="No active session found")

@app.post("/session-data")
async def update_session_data(
    data: Dict[str, Any],
    request: Request,
    current_user: UserInDB = Depends(get_current_active_user)
):
    """Update user's session data"""
    if current_user.username in active_sessions:
        active_sessions[current_user.username].user_data.update(data)
        active_sessions[current_user.username].last_activity = datetime.utcnow()
        return {"message": "Session data updated", "data": active_sessions[current_user.username].user_data}
    else:
        raise HTTPException(status_code=404, detail="No active session found")
# Fix for the /home-budget endpoint to properly handle the form submission
```

Epic 3: Backend – FastAPI Integration

Description

This focuses on implementing the FastAPI-powered backend for **PocketSmart AI**. It acts as the bridge between the frontend interface and the Gemini-based AI recommendation engine defined in gemini_utils.py. The backend is designed with modular architecture, providing clean and maintainable API endpoints for planners across Home, Party, and Jewelry domains.It also includes features like CORS setup for cross-origin requests, session handling for personalized user experience, and structured input/output models to ensure smooth AI interaction and reliable frontend rendering.

Define FastAPI Routes

Description

Home Planner (/generate-home): Accepts home-related preferences and budget to return tailored product suggestions

```python
@app.post("/home-budget")
async def plan_home_budget(
    budget_input: HomeBudgetInput,
    request: Request,
    current_user: UserInDB = Depends(get_current_active_user)
):
    """Generate home budget recommendations"""
    # Store last budget planning in session data
    if current_user.username in active_sessions:
        active_sessions[current_user.username].user_data["last_home_budget"] = {
            "timestamp": datetime.utcnow().isoformat(),
            "budget": budget_input.total_budget,
            "requirements": {
                "lights": budget_input.num_lights,
                "fans": budget_input.num_fans,
                "furniture": budget_input.num_furniture,
                "dining_tables": budget_input.num_dining_tables
            }
        }

    # Get recommendations
    result = get_home_recommendations(budget_input)

    # Save to history
    save_to_history(
        username=current_user.username,
        recommendation_type="home",
        input_data=budget_input.dict(),
        result=result
    )

    return result
```

Party Planner (/generate-party): Processes party details and guest count to recommend venue, food, and decoration items.

Jewelry Planner (/generate-jewelry): Uses text and optional image inputs to suggest jewelry based on outfit and occasion.

```python
# Update the jewelry-budget endpoint
@app.post("/jewelry-budget")
async def plan_jewelry_budget(
    total_budget: float = Form(...),
    occasion: str = Form(...),
    preferences: Optional[str] = Form(None),
    image: Optional[UploadFile] = File(None),
    request: Request = None,
    current_user: UserInDB = Depends(get_current_active_user)
):
    """Generate jewelry budget recommendations with optional outfit image"""
    # Create the budget input model
    budget_input = JewelryBudgetInput(
        total_budget=total_budget,
        occasion=occasion,
        preferences=preferences
    )

    image_path = None
    if image:
        image_path = save_upload_file(image)

    # Store last budget planning in session data
    if current_user.username in active_sessions:
        active_sessions[current_user.username].user_data["last_jewelry_budget"] = {
            "timestamp": datetime.utcnow().isoformat(),
            "budget": budget_input.total_budget,
            "occasion": budget_input.occasion,
            "has_image": image is not None
        }

    # Get recommendations
    result = get_jewelry_recommendations(budget_input, image_path)

    # Save to history with image info
    input_data = budget_input.dict()
    if image:
        input_data["image"] = image.filename

    save_to_history(
        username=current_user.username,
        recommendation_type="jewelry",
        input_data=input_data,
        result=result
    )

    return result
@app.get("/recommendation-history")
async def get_recommendation_history(
    request: Request,
    current_user: UserInDB = Depends(get_current_active_user)
):
    """Get the user's recommendation history"""
    if current_user.username not in user_recommendations:
```

```
    return {"history": []}

# Sort history by timestamp (newest first)
history = sorted(
    user_recommendations[current_user.username],
    key=lambda x: x.timestamp,
    reverse=True
)

# Convert to dict for JSON response
history_data = []
for item in history:
    history_data.append({
        "id": item.id,
        "timestamp": item.timestamp,
        "type": item.recommendation_type,
        "input": item.input_summary,
        "summary": item.result_summary
    })

return {"history": history_data}
```

User Auth (/register, /login, /logout, /token): Manages user registration, authentication, and session control.

Session Info (/session-info, /session-data): Retrieves user-specific session data for personalized AI interaction.

Modular Architecture Setup

Description

/recommendations-details: Returns detailed AI-generated product recommendations based on user budget, preferences, and selected category (Home, Party, or Jewelry).

```python
# Step 6: Add an endpoint to get details for a specific recommendation
@app.get("/recommendation-details/{recommendation_id}")
async def get_recommendation_details(
    recommendation_id: str,
    request: Request,
    current_user: UserInDB = Depends(get_current_active_user)
):
    """Get the full details of a specific recommendation"""
    if current_user.username not in user_recommendations:
        raise HTTPException(status_code=404, detail="No recommendations found")

    # Find the recommendation with the given ID
    for item in user_recommendations[current_user.username]:
        if item.id == recommendation_id:
            return {
                "id": item.id,
                "timestamp": item.timestamp,
                "type": item.recommendation_type,
                "input": item.input_summary,
                "full_result": item.full_result
            }

    raise HTTPException(status_code=404, detail="Recommendation not found")
```

CORS & Static Routing

Description

/history: Retrieves the user's past recommendation queries and results for review or re-use.

```python
# Step 7: Add a new endpoint to serve the history page
@app.get("/history", response_class=HTMLResponse)
async def history_page(request: Request, current_user: UserInDB = Depends(get_current_active_user)):
    """History page to view past recommendations"""
    return templates.TemplateResponse("history.html", {"request": request, "user": current_user})
```

Startup and main function

Description

/startup: Initializes essential application services and loads configuration settings when the FastAPI server starts.

```python
@app.on_event("startup")
async def setup_session_cleanup():
    """Background task to clean up expired sessions"""
    async def cleanup_expired_sessions():
        while True:
            current_time = datetime.utcnow()
            # Check for sessions that have been inactive for more than 30 minutes
            expired_sessions = [
                username for username, session in active_sessions.items()
                if (current_time - session.last_activity).total_seconds() > 1800  # 30 minutes
            ]

            # Remove expired sessions
            for username in expired_sessions:
                if username in active_sessions:
                    print(f"Removing expired session for {username}")
                    del active_sessions[username]

            # Wait for 5 minutes before checking again
            await asyncio.sleep(300)

    # Start the background task
    asyncio.create_task(cleanup_expired_sessions())
```

main: Entry point of the FastAPI application that starts the server using uvicorn when the script is run directly.

```python
# Main entry point
if __name__ == "__main__":
    import uvicorn
    print("Starting PocketSmart: AI Budget Planner...")
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

Epic 4: UI Development

Description

This focuses on developing a **lightweight, responsive, and intuitive frontend** for **PocketSmart AI** using **HTML**, **CSS**, and **Jinja2 templates**. The interface enables users to interact seamlessly with the system's budget-based recommendation capabilities across Home Interior, Party Planning, and Jewelry categories. It is tightly integrated with FastAPI backend endpoints to facilitate real-time input and AI-powered output rendering.The goal is to maintain a clean, user-friendly design that supports multi-category inputs, displays structured AI recommendations, and includes authentication and session-based personalization.

Develop HTML Frontend Interface

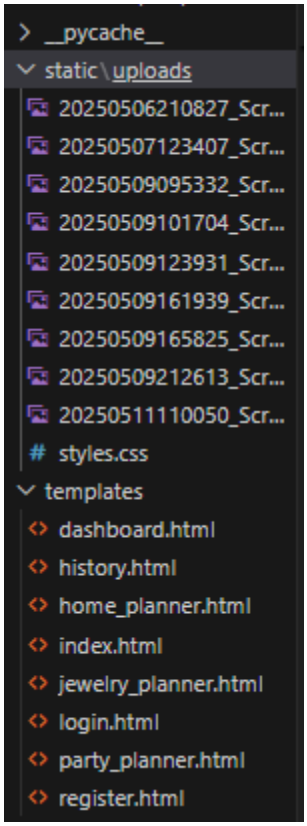
Description

Design a home page with budget input fields and category selectors (Home, Party, Jewelry)

Implement individual forms for each planner module connected to FastAPI POST routes

Display AI responses in well-structured, card-like layouts (e.g., product suggestions, item details, prices)

Templates Directory Structure:



Epic 5: Testing and Deployment

EpicComplexity: mediumDuration: 2h 0m

Description

This focuses on validating the reliability, accuracy, and contextual awareness of PocketSmart AI's recommendation engine. The system is tested using real-world user scenarios across different planning categories—Home Interiors, Party Planning, and Jewelry Suggestions—to ensure consistent performance. Optimization efforts include refining AI prompt structures, improving session handling, and ensuring a smooth UI/UX across devices.

Test with Real-World Budget Inputs Across Domains

StoryDuration: 2h 0m

Description

Conduct end-to-end testing using varied budgets and user preferences for:

Home decor setups from platforms like IKEA, Amazon

Party packages including food (Swiggy/Zomato), venues (OYO), and decor

Jewelry suggestions matched with outfit styles, occasions, and image inputs

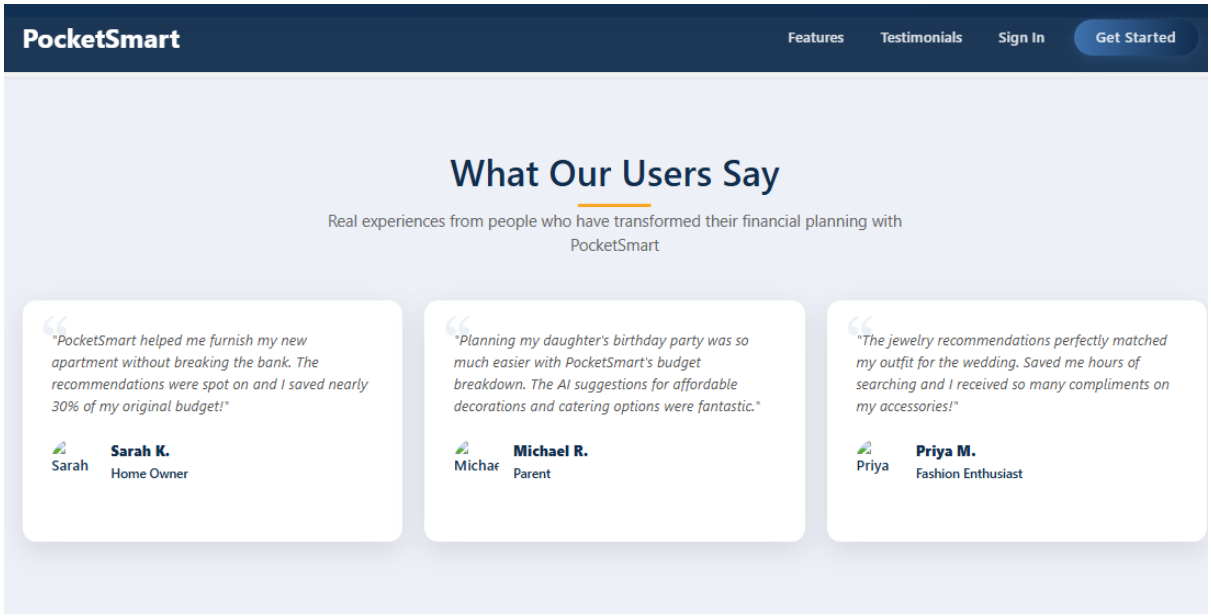
Home Page

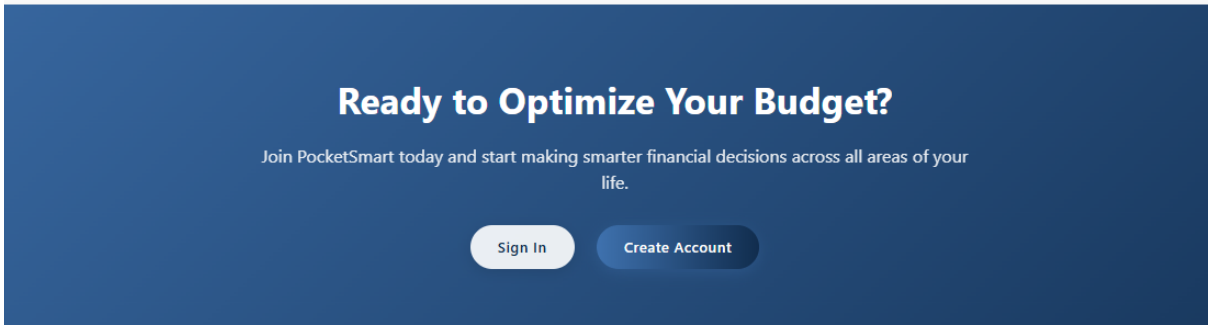
Home Page: The main landing page introducing PocketSmart AI’s features with links to planner modules and user actions.



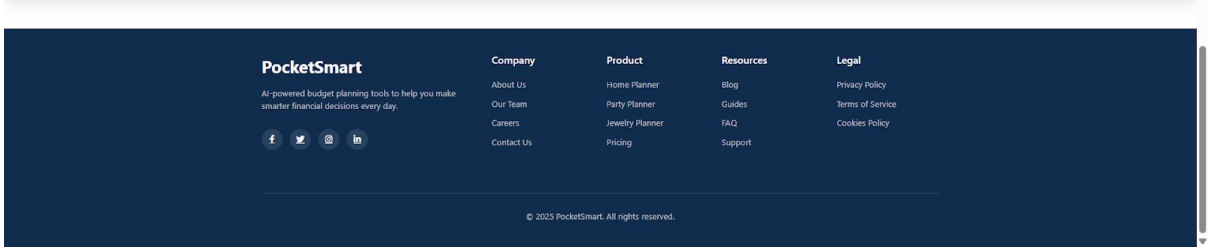
Testimonials Page

Testimonials Page: Showcases real user reviews and success stories to demonstrate the platform’s effectiveness





Footer Part



Register Page

Register Page: Allows new users to create an account to access personalized budget recommendations.

PocketSmart

AI-Powered Budget Planning

Create Your Account

Username

Choose a username

Email

Enter your email

Password

Create a strong password

Confirm Password

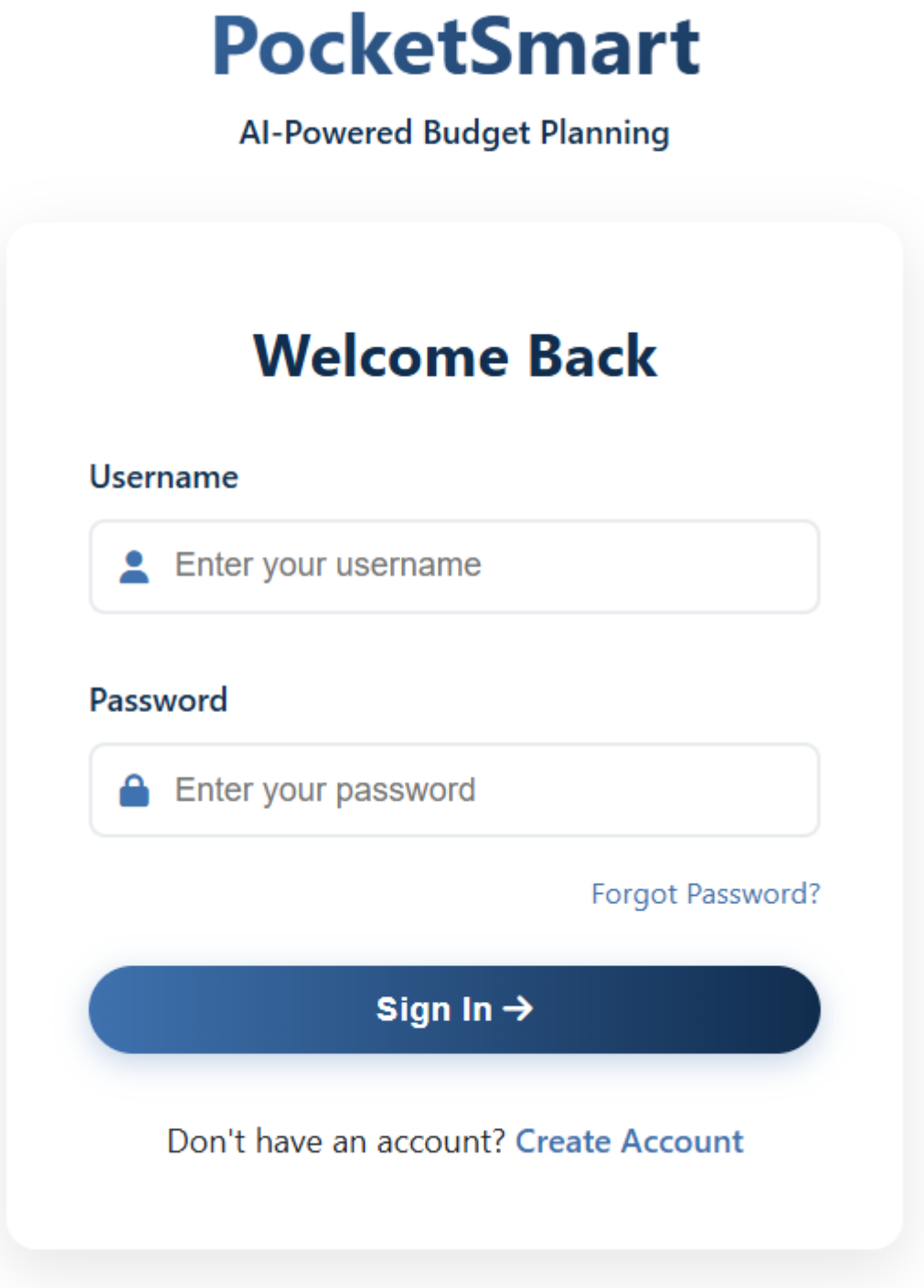
Confirm your password

Create Account

Already have an account? Sign In

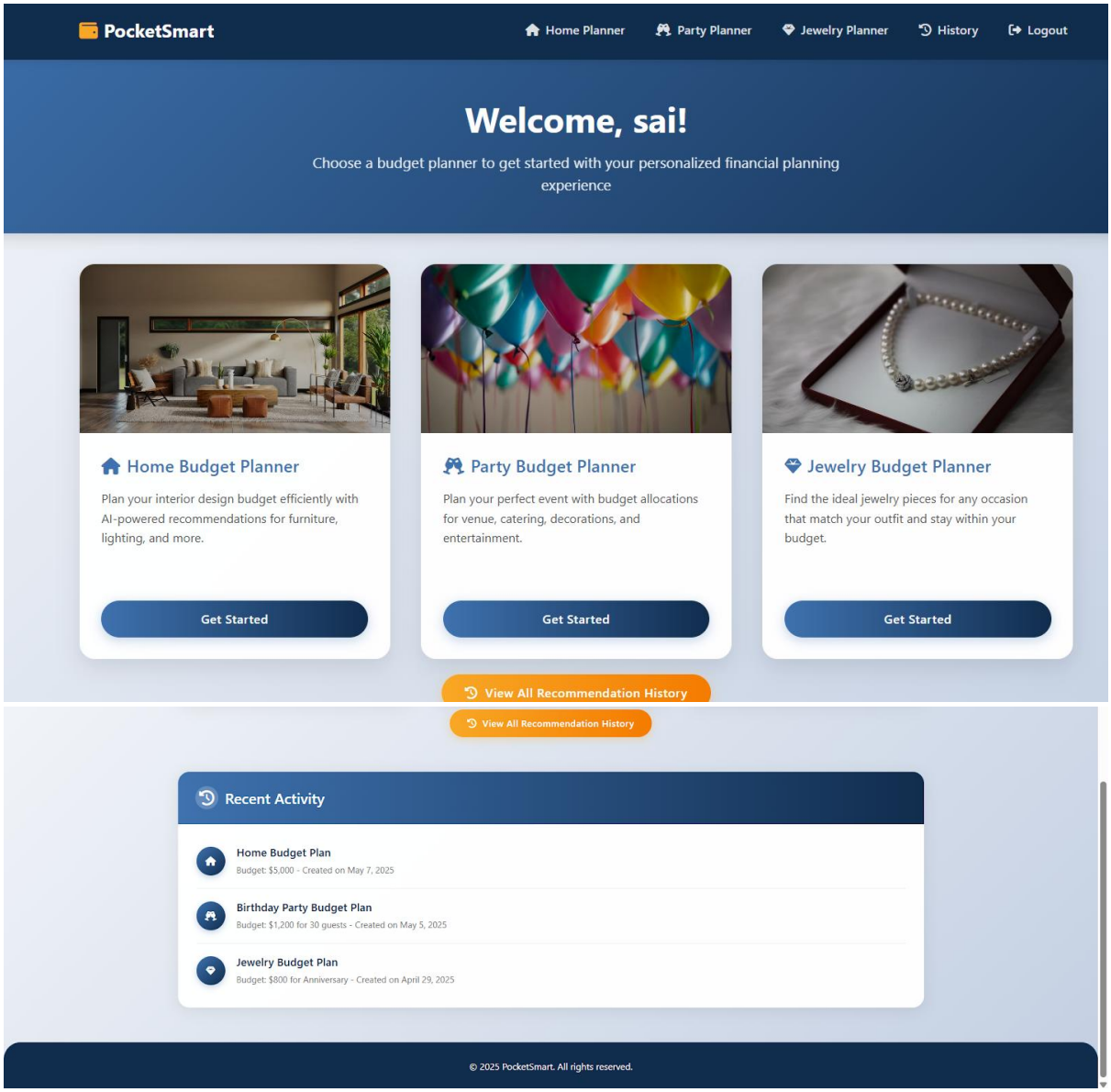
Login Page

Login Page: Authenticates existing users to access their dashboard and saved data.

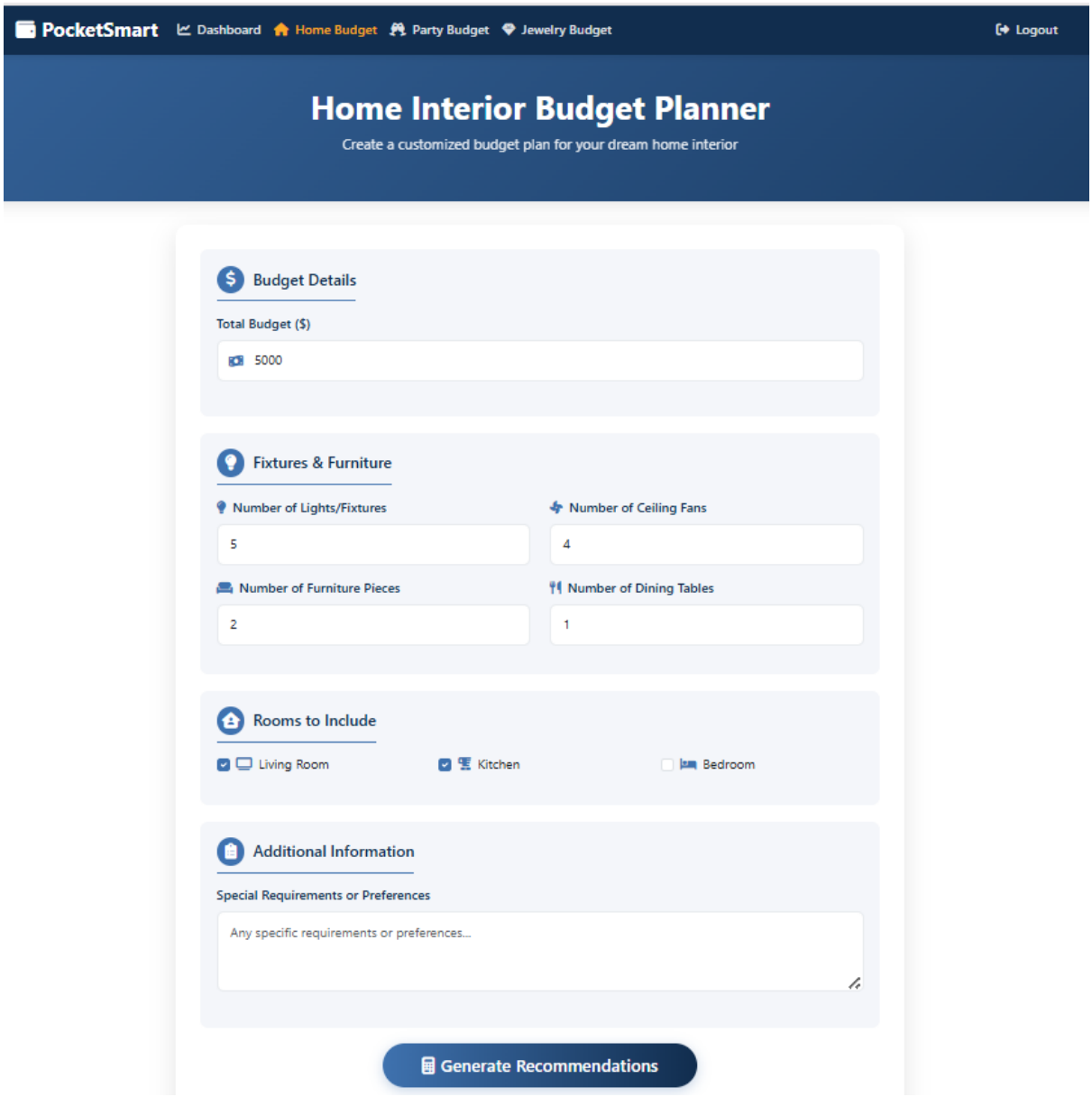


User Dashboard

User Dashboard: Displays recent recommendations, saved queries, and personalized insights for the logged-in user.



Home Interior Budget Planner Page: Lets users input home decor preferences and budget to get tailored suggestions from multiple platforms.



Home Interior Recommendations Page: Displays AI-generated product suggestions for home decor based on the user's budget and style preferences.

Generate Recommendations

Your Personalized Budget Plan

Budget Summary

Total Budget: $5000.00

Remaining Budget: $500.00

Lighting

Allocation: $1500.00

Item	Description	Price	Quantity	Shopping Links
LED Bulb (Warm White)	Energy-efficient LED bulbs for general lighting.	$100.00	5	Amazon Flipkart Ikea Myntra Ajio

Ceiling_fans

Allocation: $2000.00

Item	Description	Price	Quantity	Shopping Links
Havells Ceiling Fan	Basic, functional ceiling fan.	$500.00	4	Amazon Flipkart Ikea Myntra Ajio

Furniture

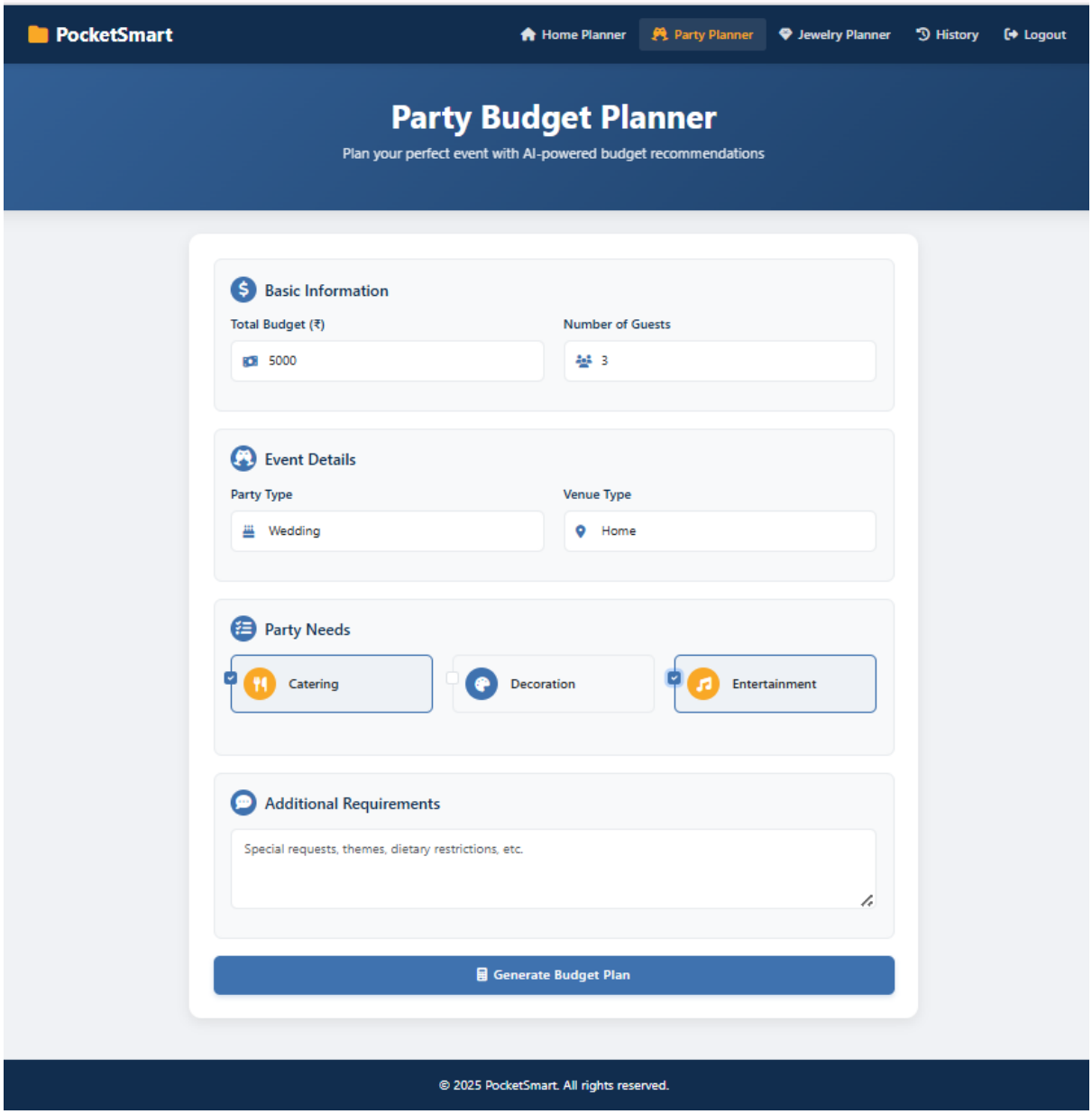
Allocation: $1000.00

Item	Description	Price	Quantity	Shopping Links
Plastic Chair	Stackable plastic chairs for kitchen or living room.	$250.00	2	Amazon Flipkart Ikea Myntra Ajio
Small Wooden Table	Simple wooden table for dining or side table.	$500.00	1	Amazon Flipkart Ikea Myntra Ajio

Additional Suggestions

- Consider purchasing used furniture for further cost savings.
- Look for sales and discounts on online marketplaces.
- Prioritize essential items and postpone non-essential purchases.

Party Budget Planner Page: Helps users plan events by suggesting venues, food, and decorations based on budget and guest count



Party Budget Recommendations Page: Shows curated party planning options, including food, venues, and decorations, matched to the provided budget.

Your Party Budget Plan

Print/Save

Budget: ₹5000.00

Venue — ₹0.00

Home — ₹0.00

Utilizing the home as the venue.

Shop on: Google, Booking, Makemytrip, Oyorooms, Nobroker

Catering — ₹2000.00

Home-cooked meal — ₹2000.00

Simple, home-cooked meal for 3 people.

Shop on: Swiggy, Zomato

Entertainment — ₹2000.00

Streaming service subscription — ₹500.00

One-time purchase of a movie or renting a few movies on a platform like Netflix, Amazon Prime Video, or Hotstar.

Shop on: Amazon, Flipkart, Bookmyshow

Board games/cards — ₹500.00

Purchase of a new board game or card game.

Shop on: Amazon, Flipkart, Bookmyshow

Music playlist — ₹0.00

Creating a celebratory playlist on Spotify or YouTube Music.

Shop on: Amazon, Flipkart, Bookmyshow

Small gift for couple — ₹1000.00

A small token of appreciation for the couple.

Shop on: Amazon, Flipkart, Bookmyshow

Contingency — ₹1000.00

Unexpected expenses — ₹1000.00

Buffer for unforeseen costs.

Shop on: Amazon, Flipkart, Google

Total Budget	**₹5000.00**
Allocated	₹5000.00
Remaining	₹0.00

Venue Suggestions

Home — ₹0.00

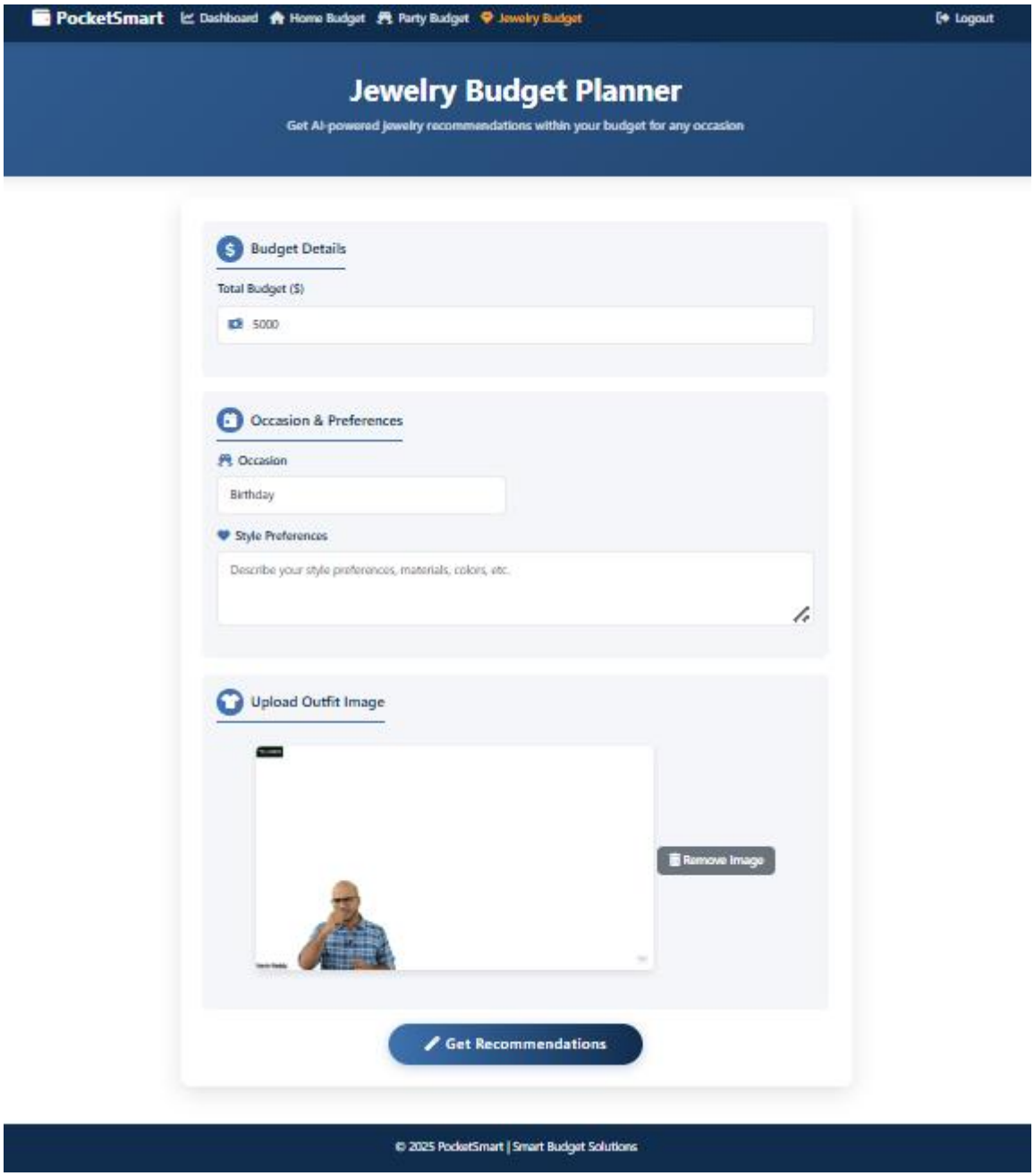
- Type: Residential
- Location: undefined
- undefined

Website View on Map

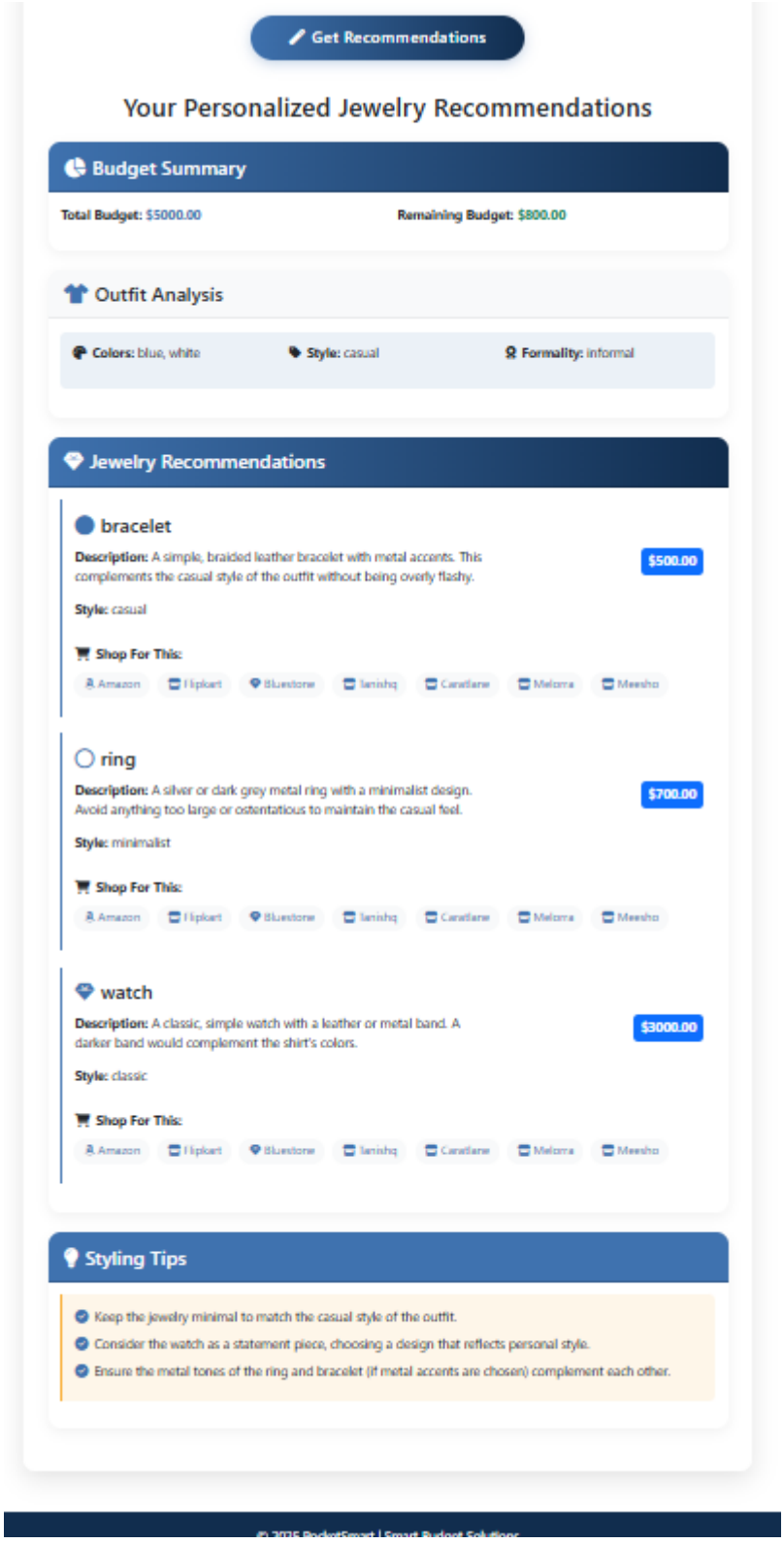
Additional Suggestions

- Consider making the meal a potluck style if comfortable with guests to reduce catering costs.
- Look for discounts or offers on streaming services or board games.
- Homemade decorations can be a cost effective alternative if you decide to add some.

Jewelry Budget Planner Page: Provides jewelry recommendations aligned with occasion and outfit style, using text and optional image inputs.



Jewelry Budget Recommendations Page: Presents personalized jewelry options tailored to the occasion, outfit, and budget, using Gemini-generated insights.



Recommendation History Page: Displays a log of the user's past recommendation queries and results, allowing easy review and reuse of previous plans.

Conclusion

Description

This is the final section of a project report that summarizes the key work completed, highlights the main outcomes and findings, and provides closure to the document by reflecting on the project's overall success and future scope.

Conclusion

Description

PocketSmart AI redefines the way individuals plan budgets for everyday lifestyle needs by delivering smart, AI-driven recommendations across **home interiors**, **party planning**, and **jewelry selection**. By combining the power of **FastAPI** with **Gemini 1.5 Flash Pro**, the system intelligently processes user inputs—budgets, preferences, and even images—to generate accurate and context-aware suggestions from multiple trusted platforms like **Amazon**, **Flipkart**, **IKEA**, **Swiggy**, **Zomato**, **OYO and more**.From backend architecture to AI integration and a responsive frontend, PocketSmart AI has been built with a focus on **efficiency**, **scalability**, and **user-centric design**. The FastAPI backend ensures modular, secure routing and robust session handling, while the Jinja2-powered frontend offers an intuitive and clean interface for users to interact with the system in real time.

Each module—from registration to recommendations—is designed to simplify decision-making and enhance the shopping or planning experience through automation and personalization. Whether it’s finding the right wall art within budget or organizing a last-minute party, PocketSmart AI acts as a **smart digital assistant**, saving time and effort while optimizing outcomes.With its seamless fusion of generative AI and web technologies, **PocketSmart AI** stands as a practical example of how **AI can enhance everyday decision-making**, bringing intelligent recommendations to users' fingertips—budget-friendly, fast, and personalized.